

Executive Summary

This report outlines the training and ONNX export outcomes for two transformer-based object detection models: RF-DETR and RT-DETR. The primary objective was to prepare these models for deployment on a Jetson Nano running TensorRT 8.2.1. Both models proved incompatible with the target hardware environment due to strict ONNX opset version limitations.

The Hardware Constraint

The target deployment environment uses TensorRT 8.2.1. This specific version has a hard compatibility ceiling: it natively supports ONNX models exported at opset 13 or lower.

Test Case 1: RF-DETR

Process and Results:

- An attempt was made to export the trained RF-DETR model to ONNX, specifically targeting opset 13 to match the TRT 8.2.1 maximum.
- The standard torch.export process failed entirely.
- The failure was triggered by a ValueError, which indicated the exporter expected a torch.nn.Module but received a different custom class structure instead.
- To troubleshoot, the model's native .export() function was triggered to verify its baseline requirements.
- The native export succeeded.

Key Finding: The native export test confirmed that RF-DETR defaults to exporting at opset 17. The conclusion is that RF-DETR requires opset ≥ 16 , making it fundamentally incompatible with TRT 8.2.1, which only supports ≤ 13 .

Test Case 2: RT-DETR

Process and Results:

- An RT-DETR model (rtdetr-l.pt) was successfully trained over 20 epochs using the Ultralytics framework on a custom 6-class dataset.
- Following training, an export to ONNX was attempted using the best.pt weights, strictly forcing opset 13.
- The opset 13 export failed. The error logs indicated that the model utilizes the aten::grid_sampler operator.
- A secondary export attempt was run using opset 17, which processed successfully and generated a valid ONNX file.

Key Finding:

The aten::grid_sampler operation required by the RT-DETR architecture was not introduced to the ONNX standard until opset 16. Because of this, the model requires a minimum of opset 16 to export correctly. As a result, it cannot be parsed by the Jetson Nano's TensorRT 8.2.1 engine.

Conclusion

Both modern architectures rely heavily on complex matrix operations and grid sampling techniques that bypass the capabilities of ONNX opset 13. Since the Jetson Nano cannot be upgraded to newer TensorRT versions that support opset 16 or 17, neither model can be utilized on this hardware.

You can find the colab notebook [here](#) for RT-DETR and full export log in annex of this report for RF-DETR

Annex

```
[2026-03-04 11:06:30] [WARNING] rf-detr - Using a different number of positional
encodings than DINOv2, which means we're not loading DINOv2 backbone weights. This
is not a problem if finetuning a pretrained RF-DETR model.
[2026-03-04 11:06:30] [WARNING] rf-detr - Using patch size 16 instead of 14, which
means we're not loading DINOv2 backbone weights. This is not a problem if finetun-
ing a pretrained RF-DETR model.
[2026-03-04 11:06:31] [INFO] rf-detr - Loading pretrain weights
[2026-03-04 11:06:31] [WARNING] rf-detr - Reinitializing detection head with 6
classes based on pretrained weights, configured for 90.
Attempting export at opset 13 (TRT 8.2.1 max)...
W0304 11:06:31.544000 1732 torch/onnx/_internal/exporter/_compat.py:125] Setting
ONNX exporter to use operator set version 18 because the requested opset_version 13
is a lower version than we have implementations for. Automatic version conversion
will be performed, which may not be successful at converting to the requested ver-
sion. If version conversion is unsuccessful, the opset version of the exported
model will be kept at 18. Please consider setting opset_version >=18 to leverage
latest ONNX features
W0304 11:06:32.281000 1732 torch/onnx/_internal/exporter/_schemas.py:455] Missing
annotation for parameter 'input' from (input, boxes, output_size: 'Sequence[int]',
spatial_scale: 'float' = 1.0, sampling_ratio: 'int' = -1, aligned: 'bool' = False).
Treating as an Input.
W0304 11:06:32.283000 1732 torch/onnx/_internal/exporter/_schemas.py:455] Missing
annotation for parameter 'boxes' from (input, boxes, output_size: 'Sequence[int]',
spatial_scale: 'float' = 1.0, sampling_ratio: 'int' = -1, aligned: 'bool' = False).
Treating as an Input.
W0304 11:06:32.285000 1732 torch/onnx/_internal/exporter/_schemas.py:455] Missing
annotation for parameter 'input' from (input, boxes, output_size: 'Sequence[int]',
spatial_scale: 'float' = 1.0). Treating as an Input.
W0304 11:06:32.287000 1732 torch/onnx/_internal/exporter/_schemas.py:455] Missing
annotation for parameter 'boxes' from (input, boxes, output_size: 'Sequence[int]',
spatial_scale: 'float' = 1.0). Treating as an Input.
[torch.onnx] Obtain model graph for `<rfdetr.main.Model object at 0x7c1e009713d0>`
with `torch.export.export(..., strict=False)`...
[torch.onnx] Obtain model graph for `<rfdetr.main.Model object at 0x7c1e009713d0>`
with `torch.export.export(..., strict=False)`... ✖
[torch.onnx] Obtain model graph for `<rfdetr.main.Model object at 0x7c1e009713d0>`
with `torch.export.export(..., strict=True)`...
[torch.onnx] Obtain model graph for `<rfdetr.main.Model object at 0x7c1e009713d0>`
with `torch.export.export(..., strict=True)`... ✖
=====
EXPORT FAILED at opset 13 - full error:
=====
Failed to export the model with torch.export. This is step 1/3 of exporting the
model to ONNX. Next steps:
- Modify the model code for `torch.export.export` to succeed. Refer to
https://pytorch.org/docs/stable/generated/exportdb/index.html for more information.
- Debug `torch.export.export` and submit a PR to PyTorch.
- Create an issue in the PyTorch GitHub repository against the *torch.export* com-
ponent and attach the full error stack as well as reproduction scripts.

## Exception summary

<class 'ValueError':> Expected `mod` to be an instance of `torch.nn.Module`, got
<class 'rfdetr.main.Model'>.

(Refer to the full stack trace above for more information.)

Trying rfdetr native export() to confirm minimum required opset...
[2026-03-04 11:06:34] [INFO] rf-detr - Exporting model to ONNX format
[2026-03-04 11:06:37] [INFO] rf-detr -
Successfully exported ONNX model: output/inference_model.onnx
```

```
[2026-03-04 11:06:37] [INFO] rf-detr - Successfully exported ONNX model to: output/inference_model.onnx
[2026-03-04 11:06:37] [INFO] rf-detr - start simplify ONNX model: output/inference_model.onnx
```

```
RF-DETR exported at opset: 17
TRT 8.2.1 max supported: 13
Compatible with Jetson Nano TRT 8.2.1: NO - incompatible
IN input [1, 3, 256, 256]
OUT dets [0, 0, 4]
OUT labels [0, 0, 7]
```

Script Used

```
cell-1a
!pip install "rfdetr[onnxexport]" onnx onnxsim --quiet
```

```
import torch
print(f"PyTorch: {torch.__version__}")
import onnx
print(f"ONNX: {onnx.__version__}")
```

```
Cell-1b
!pip install onnxscript onnx-graphsurgeon polygraphy --quiet
!pip install nvidia-pyindex --quiet
!pip install polygraphy --quiet
```

```
cell-2a
import torch
from rfdetr import RFDETRSmall
```

```
MODEL_PATH = "/content/das.pth"
IMGSZ = 256
```

```
# Load model and weights
model = RFDETRSmall(pretrain_weights=MODEL_PATH, resolution=IMGSZ)
```

```
# Attempt 1: opset 13 (TRT 8.2.1 ceiling)
print("Attempting export at opset 13 (TRT 8.2.1 max)...")
try:
```

```
    torch.onnx.export(
        model.model,
        torch.randn(1, 3, IMGSZ, IMGSZ),
        "rfdetr_opset13.onnx",
        opset_version = 13,
        input_names = ["images"],
        output_names = ["output"],
        dynamic_axes = None,
    )
    print("Succeeded - unexpected.")
```

```
except Exception as e:
    print("=" * 60)
    print("EXPORT FAILED at opset 13 - full error:")
    print("=" * 60)
    print(e)
```

```
# Attempt 2: use rfdetr's own export method to confirm minimum opset
print("\nTrying rfdetr native export() to confirm minimum required opset...")
try:
    model.export(simplify=True)
    print("\nNative export succeeded.")
    print("CONCLUSION: RF-DETR requires opset ≥16. TRT 8.2.1 supports ≤13. Incompatible.")
```

```
except Exception as e2:
    print(f"\nNative export also failed: {e2}")

cell-2b
import onnx

m = onnx.load("output/inference_model.onnx")
onnx.checker.check_model(m)

opset = m.opset_import[0].version
print(f"RF-DETR exported at opset: {opset}")
print(f"TRT 8.2.1 max supported: 13")
print(f"Compatible with Jetson Nano TRT 8.2.1: {'YES' if opset <= 13 else 'NO - in-compatible'}")

for inp in m.graph.input:
    print("IN ", inp.name, [d.dim_value for d in inp.type.tensor_type.shape.dim])
for out in m.graph.output:
    print("OUT ", out.name, [d.dim_value for d in out.type.tensor_type.shape.dim])
```